

Architecture Selection & Justification

Community Lost & Found Matching Network — Problem Statement #59

Architecture Selection

We chose Microservices Architecture for the Community Lost & Found Matching Network.

Architectural Choice

Microservices is appropriate because the system contains several distinct functions, including item management, visual and text tag extraction, geo-proximity processing, matching, notifications, and ownership verification. These functions can be separated into focused services that communicate through defined interfaces.

Reason 1 — Independent Scaling

Tag extraction and matching may require more processing than ordinary item-report management, especially when many lost and found posts are submitted. With microservices, these processing-heavy services can be scaled independently without scaling the whole system.

Reason 2 — Fault Isolation and Modularity

The system benefits from separating services because a problem in one function, such as notifications, does not have to stop core operations such as creating reports or performing matches. Individual services can also be modified without redeploying the complete system.

Security Advantage

Ownership Verification can be isolated as a dedicated service. A claimant must successfully answer the private security questions before contact details are revealed. Keeping this verification logic separate helps enforce the required access-control step and protects sensitive contact information.

Performance Benefit

The architecture can improve performance by allowing the Tag Extraction and Matching services to scale independently when the number of uploaded items or matching operations increases. This helps avoid making the entire application larger just to handle peaks in these particular workloads.

Selected components: Item Management Service, Tag Extraction Service, Matching Service, Geo-Location Service, Notification Service, Ownership Verification Service, and Item Database.

Uml Component Diagram

